Python Laboratory Code Repository

Source Code Compendium • Pure Code Listings (Without Program Outputs)

UNIT-I: EIGEN VALUES & EIGENVECTORS

Program 1: Eigenvalues and Eigenvectors with Visualization

```
#4.Write a Python Program to Find the Real & Complex Eigen Values and
#Eigen Vectors of a User-defined Matrix with Visualization
import numpy as np
import cmath
import matplotlib.pyplot as plt

n=int(input("Enter the order of the matrix"))
A=[]
for i in range(n):
    row=list(map(float, input().split()))
    A.append(row)
print("A=", A)
e, V=np.linalg.eig(A)
print("Eigen values of A:", e)
print("Eigen vector of A(column-wise):", V)
for i in range(n):
    v=V[:,i]
    Av=A@v
    print("Av=", Av)
    plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1, label="x", color='blue')
    plt.quiver(0, 0, -v[0], -v[1], angles='xy', scale_units='xy', scale=1, label="x",
color='blue')
    plt.quiver(0, 0, Av[0], Av[1], angles='xy', scale_units='xy', scale=1, label="Ax",
color='red')
    plt.quiver(0, 0, -Av[0], -Av[1], angles='xy', scale_units='xy', scale=1, label="Ax", color='r
ed')
plt.xlim(-6,6)
plt.ylim(-6,6)
plt.axhline(0,color="black")
plt.axvline(0,color="black")
plt.title("Visualization of Eigenvectors of the Given Matrix")
plt.grid(True)
plt.legend()
plt.show()
```

NEWTON-RAPHSON METHOD & TRANSCENDENTAL EQUATIONS

Program 2: Transcendental Equation Root Finding (Newton-Raphson Method)

```python
#3.Write a Python Program to find the root of a User-defined Transcendental Equation
#Equation with Visualization using Newton Raphson Method

import numpy as np
import matplotlib.pyplot as plt

x = float(input("Enter the initial guess x0: "))
function = input("Enter the function (use x as the variable): ")
derivative = input("Enter the derivative of the function: ")
n = abs(int(input("Enter the number of iterations: ")))

def f(x):
    return eval(function)

def df(x):
    return eval(derivative)

print("-" * 120)
print(f"|{'S.NO':^6}|{'x':^16}|{'f(x)':^20}|{'f\'(x)':^20}|{'x_new':^16}|")
print("-" * 120)

for i in range(n):
    fx = f(x)
    dfx = df(x)

    if dfx == 0:
        print("Derivative is zero. Newton-Raphson method fails.")
        break

    x_new = x - fx / dfx

    print(f"|{i+1:<6}|{x:<16.6f}|{fx:<20.6f}|{dfx:<20.6f}|{x_new:<16.6f}|")
    print("-" * 120)

    x = x_new

print("Approximate root =", x)

#Plot
x=np.linspace(1,3,400)
y=f(x)

plt.plot(x,y,label='$f(x)= x *log(x) - 1.2$')
plt.axhline(0, color="black")
plt.grid(True)
plt.legend()
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Function graph")
plt.show()
```

GAUSS-SEIDEL ITERATIVE METHOD

Program 3: System of Linear Equations (Gauss-Seidel Iterative Method)

```python
#Write a Python program to solve a system of linear equations using Gauss-Seidel iterative method
#and visualize the convergence of the solution.

import numpy as np
import matplotlib.pyplot as plt

A = np.array([[8, -3, 2],
              [4, 11, -1],
              [6, 3, 12]], dtype=float)
B = np.array([20,33,36], dtype=float)
x = np.zeros(3)
n = 10

iterations = []
x1_values = []
x2_values = []
x3_values = []

print("-" * 70)
print(f"{'k':<10} {'x[0]':<15} {'x[1]':<15} {'x[2]':<15}")
print("-" * 70)

for k in range(n):

    x[0] = (B[0] - A[0,1]*x[1] - A[0,2]*x[2]) / A[0,0]

    x[1] = (B[1] - A[1,0]*x[0] - A[1,2]*x[2]) / A[1,1]

    x[2] = (B[2] - A[2,0]*x[0] - A[2,1]*x[1]) / A[2,2]

    iterations.append(k + 1)
    x1_values.append(x[0])
    x2_values.append(x[1])
    x3_values.append(x[2])

    print(f"{k+1:<10} {x[0]:<15.6f} {x[1]:<15.6f} {x[2]:<15.6f}")
    print("-" * 70)

print("Gauss-Seidel Solution:")
print(x)

# --------------- VISUALIZATION --------------- #

plt.figure(figsize=(8, 5))
plt.plot(iterations, x1_values, marker='o', label='x1')
plt.plot(iterations, x2_values, marker='s', label='x2')
plt.plot(iterations, x3_values, marker='^', label='x3')
plt.xlabel("Iteration")
plt.ylabel("Solution values")
plt.title("Convergence of Gauss-Seidel Method")
plt.xticks(iterations)
plt.grid(True)
plt.legend()
plt.show()
```

EXACT & NON-EXACT DIFFERENTIAL EQUATIONS

Program 4: Non-Exact Differential Equations & Integrating Factor

```python
#4.Write a Python program to check whether a given differential equation is nonexact,
#find a suitable integrating factor, obtain its solution, and visualize the
#solution curves:(y^4+2*y)dx+(x*y^3+2*y^4-4*x)dy=0.

import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

x, y = sp.symbols('x y')

M = y**4+2*y
N = x*y**3+2*y**4-4*x

My = sp.diff(M, y)
Nx = sp.diff(N, x)

print("M =", M)
print("N =", N)
print("∂M/∂y =", My)
print("∂N/∂x =", Nx)

if My == Nx:
    print("The differential equation is EXACT.")

else:
    print("The differential equation is NOT EXACT.")

    IF = sp.exp(sp.integrate((Nx - My)/M, y))

    print("Integrating Factor =", sp.simplify(IF))

    M1 = sp.simplify(IF * M)
    N1 = sp.simplify(IF * N)

    F = sp.integrate(M1, x)
    g_prime = sp.simplify(N1 - sp.diff(F, y))
    g = sp.integrate(g_prime, y)
    F = sp.simplify(F + g)

    print("Solution:")
    print(F, "= C")

# Visualization

F_func = sp.lambdify((x, y), F, 'numpy')

x_vals = np.linspace(-2, 2)
y_vals = np.linspace(-2, 2)
X, Y = np.meshgrid(x_vals, y_vals)
Z = F_func(X, Y)
C_values = [-3,-2,-1,0,1,2,3]
contours = plt.contour(X, Y, Z, levels=C_values)

plt.clabel(contours, inline=True, fontsize=10)
plt.axhline(0, color='red')
plt.axvline(0, color='blue')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Solution Curves of Exact Differential Equation")
plt.grid(True)
plt.show()
```